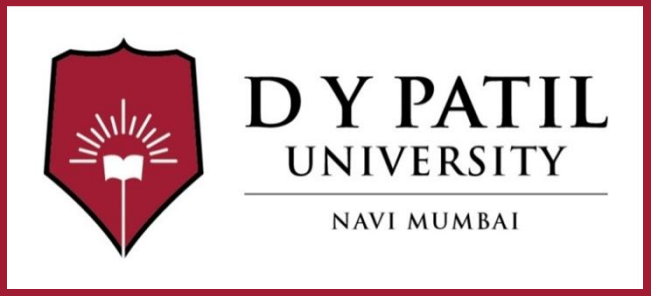
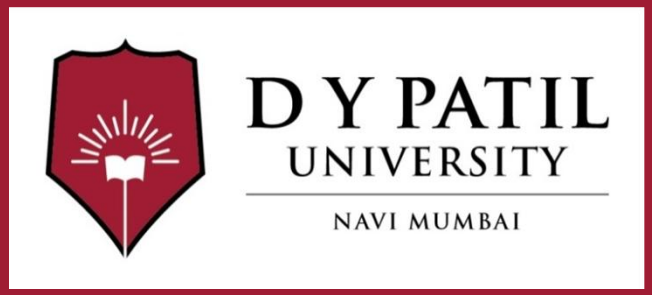


Subject Name :

Unit No: 5

Unit Name :Introduction to Object Oriented Concepts

Faculty Name :Swati Kadu



Unit No.:5

Faculty Name :Swati Kadu

Index

Lecture No: 1

Introduction to Object Oriented Concepts

Object Oriented Programming empowers developers to build modular, maintainable and scalable applications.

OOP is a way of organizing code that uses objects and classes to represent real-world entities and their behavior.

In OOP, object has attributes thing that has specific data and can perform certain actions using methods.

- Organizes code into classes and objects.
- Supports encapsulation to group data and methods together.
- Enables inheritance for reusability and hierarchy.
- Allows polymorphism for flexible method implementation.
- Improves modularity, scalability and maintainability.

Python Scopes and Namespaces

1. Namespaces

A namespace ensures that names are unique and don't collide. Python maintains three main types:

- **Built-in Namespace:** Contains built-in functions (like `print()` and `len()`) and exceptions. It is created when the Python interpreter starts.
- **Global Namespace:** Contains names defined at the main body of a script or module. It lasts until the script terminates.
- **Local Namespace:** Created when a function is called; it contains arguments and variables defined inside that function. It is deleted when the function returns.

2. Python Scopes

When you reference a variable, Python searches for it in a very specific order. This is known as the **LEGB Rule**:

1. **Local:** Names assigned within a function (or lambda) and not declared global.
2. **Enclosing:** Names in the local scope of any and all enclosing functions (relevant for nested functions).
3. **Global:** Names assigned at the top-level of a module file, or declared global in a `def` within the file.
4. **Built-in:** Names preassigned in the built-in names module (e.g., `open`, `range`).

3. Scope Modification: `global` and `nonlocal`

Sometimes you need to reach out of your current scope to modify a variable. Python provides two keywords for this:

The `global` Keyword

Used inside a function to modify a variable defined in the global (top-level) namespace.

Python Code:

```
x = 10
def change():
    global x
    x = 20  # Changes the global x

change()
print(x) # Output: 20
```

The `nonlocal` Keyword

Used in nested functions to modify a variable in the **enclosing** (outer function) scope, rather than the global scope.

Python Code:

```
def outer():
    x = "local"
    def inner():
        nonlocal x
        x = "nonlocal" # Changes the x in outer()
    inner()
    print(x)

outer() # Output: nonlocal
```

D Y PATIL
UNIVERSITY
NAVI MUMBAI

4. Lifetime of a Namespace

- **Built-in:** Lasts as long as the interpreter is running.
- **Global:** Lasts until the module is unloaded (usually when the program ends).
- **Local:** Lasts until the function returns or raises an unhandled exception.

Class

A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have. Some points on Python class:

- Classes are created by keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be accessed using the dot (.) operator.
- **Example:** Myclass.Myattribute

syntax:

```
class className:
        '''documentation string'''
        variables:Instance, static and local variables
        methods:Instance, static, class methods
```

-->Documentation string represents description of the class. Within the class doc string is always optional. We can get doc string by using following 2-ways.

```
1).print(classname.__doc__)
2).help(classname)
```

Creating a Class

Here, class keyword indicates that we are creating a class followed by name of the class (Dog in this case).

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

Explanation:

- class Dog: creates a class named Dog, which acts as a blueprint for dog objects.
- species is a class attribute, meaning it is shared by all instances of the class.
- __init__() is a constructor method that runs automatically when a new object is created. It is used to initialize object data.
- self refers to the current object, allowing each object to store and access its own data.
- self.name and self.age are instance attributes, unique to each Dog object created from the class.

Objects

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data. An object consists of:

- **State:** It is represented by the attributes and reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object and reflects the response of an object to other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

Creating Object

Creating an object in Python involves instantiating a class to create a new instance of that class. This process is also referred to as object instantiation.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)
print(dog1.species)
```

Output

```
Buddy
Canine
```

Explanation:

- **dog1 = Dog("Buddy", 3):** Creates an object of the Dog class with name as "Buddy" and age as 3.
- **dog1.name:** Accesses the instance attribute name of the dog1 object.
- **dog1.species:** Accesses the class attribute species of the dog1 object.

self Variable:

-->self is a default variable which is always pointing to current object(like this keyword in java).

-->By using self we can access instance variables and instance methods of object.

Note:

---

1).self should be first parameter inside constructor.

```
def __init__(self):
```

2).self should be first parameter inside instance methods.

```
def talk(self):
```

Python __init__() Method

The __init__() Method

- All classes have a built-in method called __init__(), which is always executed when the class is being initiated.
- The __init__() method is used to assign values to object properties, or to perform operations that are necessary when the object is being created.

Note: The __init__() method is called automatically every time the class is being used to create a new object.

Why Use __init__()?

- Without the __init__() method, you would need to set properties manually for each object
- Using __init__() makes it easier to create objects with initial values:

Default Values in __init__()

- You can also set default values for parameters in the __init__() method
- Multiple Parameters
- The __init__() method can have as many parameters as you need
- `self` is a **reference to the current object (instance) of a class**. It is used to **access variables and methods that belong to that object**.
- `self` allows each object to **store and access its own data**.

Delete Objects

You can delete objects by using the `del` keyword:

Example

Delete the p1 object:

```
del p1
```

Multiple Objects

You can create multiple objects from the same class:

Example

Create three objects from the MyClass class:

```
p1 = MyClass()
p2 = MyClass()
p3 = MyClass()

print(p1.x)
print(p2.x)
print(p3.x)
```

The pass Statement

`class` definitions cannot be empty, but if you for some reason have a `class` definition with no content, put in the `pass` statement to avoid getting an error.

Example

```
class Person:
  pass
```

Example:

```
class Student:
  '''Developed by Swati for python demo'''
  def __init__(self):
   self.name="Swati"
   self.age=30
   self.marks=100
  def talk(self):
   print("Hello I Am:",self.name)
   print("My Age Is:",self.age)
   print("My Marks Are:",self.marks)
s=Student()
print("Student Name:",s.name)
print("Student Age:",s.age)
print("Student Marks:",s.marks)
```

Output:

```
Student Name: Swati
Student Age: 30
Student Marks: 100
```

```
class Employee:
   'Common base class for all employees'
   empCount = 0

   def __init__(self, name, salary):
      self.name = name
      self.salary = salary
      Employee.empCount += 1

   def displayCount(self):
     print "Total Employee %d" % Employee.empCount

   def displayEmployee(self):
      print "Name : ", self.name,  ", Salary: ", self.salary

"This would create first object of Employee class"
emp1 = Employee("Zara", 2000)
"This would create second object of Employee class"
emp2 = Employee("Manni", 5000)
emp1.displayEmployee()
emp2.displayEmployee()
print "Total Employee %d" % Employee.empCount
```

Instead of using the normal statements to access attributes, you can use the following functions −

- The **getattr(obj, name[, default])** − to access the attribute of object.
- The **hasattr(obj,name)** − to check if an attribute exists or not.
- The **setattr(obj,name,value)** − to set an attribute. If attribute does not exist, then it would be created.
- The **delattr(obj, name)** − to delete an attribute.

```
hasattr(emp1, 'age')    # Returns true if 'age' attribute exists
getattr(emp1, 'age')    # Returns value of 'age' attribute
setattr(emp1, 'age', 8) # Set attribute 'age' at 8
delattr(empl, 'age')    # Delete attribute 'age'
```

Built-In Class Attributes

Every Python class keeps following built-in attributes and they can be accessed using dot operator like any other attribute −

- **__dict__** − Dictionary containing the class's namespace.
- **__doc__** − Class documentation string or none, if undefined.
- **__name__** − Class name.
- **__module__** − Module name in which the class is defined. This attribute is "__main__" in interactive mode.
- **__bases__** − A possibly empty tuple containing the base classes, in the order of their occurrence in the base class list.

Destroying Objects (Garbage Collection)

- Python deletes unneeded objects (built-in types or class instances) automatically to free the memory space. The process by which Python periodically reclaims blocks of memory that no longer are in use is termed Garbage Collection.
- Python's garbage collector runs during program execution and is triggered when an object's reference count reaches zero. An object's reference count changes as the number of aliases that point to it changes.
- An object's reference count increases when it is assigned a new name or placed in a container (list, tuple, or dictionary). The object's reference count decreases when it's deleted with del, its reference is reassigned, or its reference goes out of scope. When an object's reference count reaches zero, Python collects it automatically

```
a = 40      # Create object <40>
b = a       # Increase ref. count  of <40>
c = [b]     # Increase ref. count  of <40>
del a       # Decrease ref. count  of <40>
b = 100     # Decrease ref. count  of <40>
c[0] = -1   # Decrease ref. count  of <40>
```

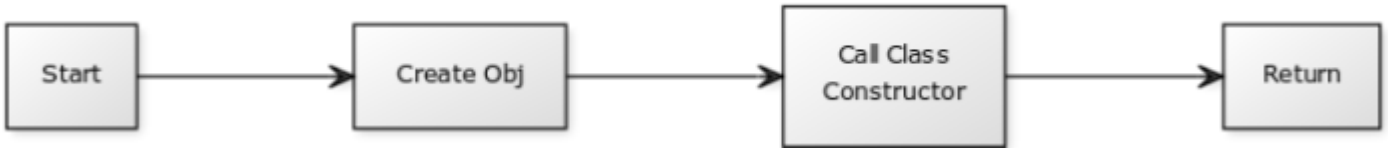
- You normally will not notice when the garbage collector destroys an orphaned instance and reclaims its space. But a class can implement the special method __del__(), called a destructor, that is invoked when the instance is about to be destroyed. This method might be used to clean up any non memory resources used by an instance.

This __del__() destructor prints the class name of an instance that is about to be destroyed

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __del__(self):
        class_name = self.__class__.__name__
        print(class_name, "destroyed")

pt1 = Point()
pt2 = pt1
pt3 = pt1

print(id(pt1), id(pt2), id(pt3))  # prints ids of the objects

del pt1
del pt2
del pt3
```

Constructor

- Constructor is a special method in Python.
- The name of the constructor should be __init__(self).
- Constructor will be executed automatically at the time of object creation.
- The main purpose of constructor is to declare and initialize instance variables.
- Per object constructor will be executed once.
- Constructor can take at least one argument(at least self).
- Constructor is optional and if we are not providing any constructor python will provide default constructor.
- Each time an object is created a method is called. That methods is named the **constructor**.



Program to demonstrate constructor will execute only once per object:

```
class Test:
        def __init__(self):
                print("Constructor exeuction...")
                def m1(self):
                        print("Method execution...")
                        t1=Test()
                        t2=Test()
                        t3=Test()
                        t1.m1()
```

Output:

```
Constructor exeuction...
Constructor exeuction...
Constructor exeuction...
Method execution...
```

```
class Student:
    def __init__(self,x,y,z):
        self.name=x
        self.rollno=y
        self.marks=z
    def display(self):
        print("Student Name:{}\nRollno:{}\nMarks:{}".format(self.name,self.rollno,self.marks))
s1=Student("Swati",101,80)
s1.display()
s2=Student("Prachi",102,100)
s2.display()
```

Output:

```
Student Name:Swati
Rollno:101
Marks:80
Student Name:Prachi
Rollno:102
Marks:100
```

How to create list of objects in constructor:

```
class Movie:
  def __init__(self,name,hero,heroine,rating):
   self.name=name
   self.hero=hero
   self.heroine=heroine
   self.rating=rating

  def info(self):
   print("Movie Name:",self.name)
   print("Hero Name:",self.hero)
   print("Heroine Name:",self.heroine)
   print("Movie Rating :",self.rating)
   print()
movies=[Movie('Spider','John','Alice',30),Movie('Bahubali','Prabhas','Anusha',70),
    Movie('Godzilla','Jimmy','Katty',90)]
for movie in movies:
  movie.info()
```

```
Movie Name: Spider
Hero Name: John
Heroine Name: Alice
Movie Rating : 30

Movie Name: Bahubali
Hero Name: Prabhas
Heroine Name: Anusha
Movie Rating : 70

Movie Name: Godzilla
Hero Name: Jimmy
Heroine Name: Katty
Movie Rating : 90
```

without constructor

```
class Movie:
  def info(self):
    print("Movie Name:",self.name)
    print("Hero Name:",self.hero)
    print("Heroine Name:",self.heroine)
    print("Movie Rating :",self.rating)
    print()
m=Movie()
m.name="Spider"
m.hero="John"
m.heroine="alice"
m.rating="90"
m.info()
```

Output:

```
Movie Name: Spider
Hero Name: John
Heroine Name: alice
Movie Rating : 90
```

Difference between Methods and Constructors:

Method	Constructor
1. Name of method can be any name.	1. Constructor name should be always __init__
2. Method will be executed, if we call that method.	2. Constructor will be executed automatically at the time of object creation.
3. Per object, method can be called any number of times.	3. Per object, Constructor will be executed only once.
4. Inside method we can write and initialize instance variables.	4. Inside Constructor we have to declare business logic.

Types of Variables:

Inside Python class 3 types of variables are allowed:

1. Instance Variables (Object Level Variables)
2. Static Variables (Class level Variables)
3. Local Variables (Method Level Variables)

1. Inside constructor by using self variable.

We can declare instance variables inside a constructor by using self keyword.
Once we create the object, automatically these variables will be added to the object.

Ex:

```
1)  class Employee:
2)          def __init__(self):
3)                  self.eno=100
4)                  self.ename="Mahesh"
5)  e=Employee()
6)  print(e.__dict__)
```

output:
{'eno':100,'ename':'Mahesh'}

2. Inside Instance Method by using self variable.

- We can also declare instance variables inside instance method by using self variable. If any instance variable declared inside instance method, that instance variable will be added once we call that method.

```
class Test:
    def __init__(self):
        self.a=10
        self.b=20
    def m1(self):
        self.c=30
t=Test()
t.m1()
print(t.__dict__)
```

Output:

```
{'a': 10, 'b': 20, 'c': 30}
```

3. Outside of the class by using object reference variable.

```
class Test:
    def __init__(self):
        self.a=10
        self.b=20
    def m1(self):
        self.c=30
t=Test()
t.m1()
t.d=40
print(t.__dict__)
```

Output:

{'a': 10, 'b': 20, 'c': 30, 'd': 40}

How to access Instance Variables

We can access instance variables with in the class by using self variable and outside of the class by using object reference.

```
class Test:
    def __init__(self):
        self.a=10
        self.b=20
    def m1(self):
        self.c=30
t=Test()
t.m1()
t.d=40
Print(t.a,t.b,t.c)
```

Output:

10 20 30

Static Variables (Class Variables)

- Declared **inside the class**, but **outside any method**.
- **Shared** by all instances of the class.
- Changing the value via the class name affects all instances (unless overridden in an instance).
- **Example:**

```
class Car:
    wheels = 4  # Class variable (shared by all objects)

    def __init__(self, brand):
        self.brand = brand  # Instance variable

# Accessing class variable
car1 = Car("Toyota")
car2 = Car("Honda")

print(car1.wheels)  # 4
print(car2.wheels)  # 4

Car.wheels = 6  # Change for all instances
print(car1.wheels)  # 6
print(car2.wheels)  # 6
```

How to access static variables:

1. inside constructor: by using either self or classname
2. inside instance method: by using either self or classname
3. inside class method: by using either cls variable or classname
4. inside static method: by using classname
5. From outside of class: by using either object reference or classname.

```
class Test:
  a=10
  def __init__(self):
   print(self.a)
   print(Test.a)
  def m1(self):
   print(Test.a)
   print(self.a)
  @classmethod
  def m2(cls):
   print(Test.a)
   print(cls.a)
  @staticmethod
  def m3():
   print(Test.a)
t=Test()
print(Test.a)
t.m1()
t.m2()
t.m3()
```

Where we can modify the value of static variable:

Anywhere either with in the class or outside of class we can modify by using classname.
But inside class method, by using cls variable.

```
1)  class Test:
2)          a=777
3)          @classmethod
4)          def m1(cls):
5)                  cls.a=333
6)          @staticmethod
7)          def  m2():
8)                  Test.a=999
9)  print(Test.a)
10) Test.m1()
11) print(Test.a)
12) Test.m2()
13) print(Test.a)
```

output:
777
333
999

******

If we change the value of static variable by using either self or object reference variable:

If we change the value of static variable by using either self or object reference variable, then the value of static variable won't be changed,just a new instance variable with that name will be added to that particular object.

Local variables:

Sometimes to meet temporary requirements of programmer,we can declare variables inside a method directly,such type of variables are called local variable or temporary variables.

Local variables will be created at the time of method execution and destroyed once method completes.

Local variables of a method cannot be accessed from outside of method.

Example:

```
class Test:
    def m1(self):
        a=1000
        print(a)
    def m2(self):
        b=2000
        print(b)
t=Test()
t.m1()
t.m2()
```

Output:

```
1000
2000
```

Four Pillars of OOP in Python

1 Inheritance

Inheritance allows a class (child class) to acquire properties and methods of another class (parent class). It supports hierarchical classification and promotes code reuse.

2. Polymorphism

Polymorphism in Python means "same operation, different behavior." It allows functions or methods with the same name to work differently depending on the type of object they are acting upon.

The flowchart below represents the different types of polymorphism in Python, showing how a single interface can exhibit multiple behaviors at compile-time and run-time.

3. Encapsulation

Encapsulation is the bundling of data (attributes) and methods (functions) within a class, restricting access to some components to control interactions. A class is an example of encapsulation as it encapsulates all the data that is member functions, variables, etc.

4. Data Abstraction

Abstraction hides the internal implementation details while exposing only the necessary functionality. It helps focus on "what to do" rather than "how to do it."

Encapsulation

Encapsulation is one of the core concepts of Object Oriented Programming (OOP).

- The idea of encapsulation is to bind the data members and methods into a single unit.
- Helps in better maintainability, readability and usability as we do not need to explicitly pass data members to member methods
- Helps maintain data integrity by allowing validation and control over the values assigned to variables.
- It helps in achieving abstraction. A class can hide the implementation part and discloses only the functionalities required by other classes which allows later changes to representations or implementations without impacting the codes that uses this class.

This example shows encapsulation by keeping __salary variable private inside Employee class. It cannot be accessed directly from outside the class.

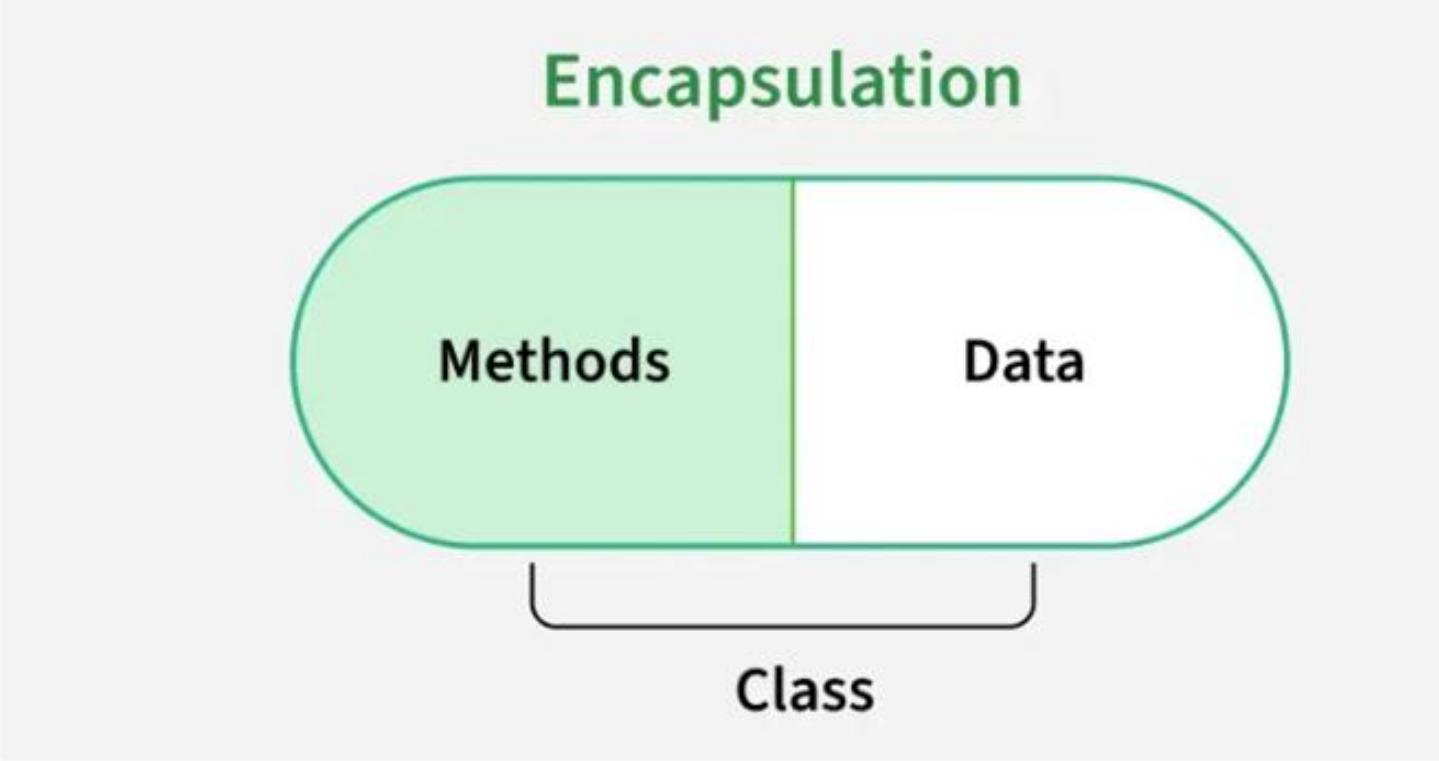
```
class Employee:
    def _ _init_ _(self, name, salary):
        self.name = name          # public
    attribute
        self._ _salary = salary    # private
    attribute

emp = Employee("Fedrick", 50000)
print(emp.name)
print(emp._ _salary)
```

Output:

Fedrick
ERROR!
Traceback (most recent call last):
File "<main.py>", line 8, in <module>
AttributeError: 'Employee' object has no attribute '__salary'

Explanation:

- **self.name = name:** Public attribute, can be accessed directly.
- **self.__salary = salary:** Private attribute, cannot be accessed directly.
- **print(emp.name):** Prints "Fedrick" because name is public.
- **print(emp.__salary):** Raises an error because __salary is private and hidden.

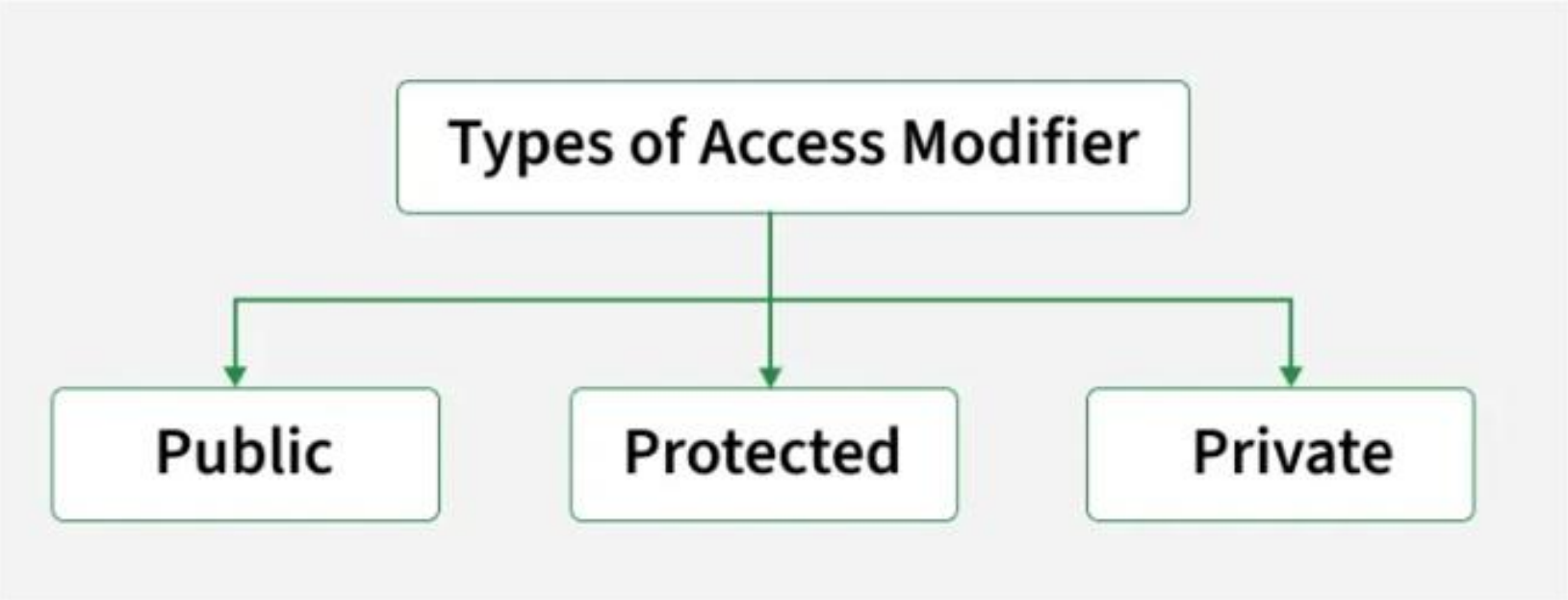


Why do we need Encapsulation?

- Protects data from unauthorized access and accidental modification.
- Controls data updates using getter/setter methods with validation.
- Enhances modularity by hiding internal implementation details.
- Simplifies maintenance through centralized data handling logic.
- Reflects real-world scenarios like restricting direct access to a bank account balance.

Access Specifiers

Access specifiers define how class members (variables and methods) can be accessed from outside the class. They help in implementing encapsulation by controlling the visibility of data. There are three types of access specifiers:



1. Public Members

- Public members are variables or methods that can be accessed from anywhere inside the class, outside the class or from other modules. By default, all members in Python are public. They are defined without any underscore prefix (e.g., self.name).
- **Example:** This example shows how a public attribute (name) and a public method (display_name) can be accessed from outside the class using an object.
- **Example:**

```
class Employee:
    def _ _init_ _(self, name):
        self.name = name   # public attribute
    def display_name(self):   # public method
        print(self.name)
emp = Employee("John")
emp.display_name()   # Accessible
print(emp.name)      # Accessible
```

Output:

```
John
John
```

Explanation:

- **self.name:** Declared without underscores, so it is public.
- **display_name():** Public method that prints the value of the public attribute.
- **emp.name:** Directly accessed from outside the class, showing public members are fully accessible

Protected members

- Protected members are variables or methods that are intended to be accessed only within the class and its subclasses. They are not strictly private but should be treated as internal.
- In Python, **protected members are defined with a single underscore prefix (e.g., self._name).**
- **Example:** This example shows how a protected attribute **(_age)** can be accessed within a subclass, demonstrating that protected members are meant for use within the class and its subclasses.

```
class Employee:
    def _ _init_ _(self, name, age):
        self.name = name       # public
        self._age = age        # protected
class SubEmployee(Employee):
    def show_age(self):
        print("Age:", self._age)   # Accessible in subclass
emp = SubEmployee("Ross", 30)
print(emp.name)        # Public accessible
emp.show_age()         # Protected accessed through subclass
```

Explanation:

self._age: Defined with a single underscore, marking it as protected.

SubEmployee: Inherits from Employee and can access _age directly.

Protected members should not be accessed outside the class hierarchy, but Python does not enforce this rule strictly.

Private members

- Private members are variables or methods that cannot be accessed directly from outside the class. They are used to restrict access and protect internal data.
- **In Python, private members are defined with a double underscore prefix (e.g., self._ _salary).**
- Python uses name mangling, where the interpreter internally renames the variable (for eg, _ _salary becomes _ClassName_ _salary).
- This discourages direct access from outside the class, although it does not create strict privacy like other languages.

Example: This example shows how a private attribute (__salary) is accessed within the class using a public method, demonstrating that private members cannot be accessed directly from outside the class.

```
class Employee:
    def _ _init_ _(self, name, salary):
        self.name = name          # public
        self._ _salary = salary    # private

    def show_salary(self):
        print("Salary:", self._ _salary)

emp = Employee("Robert", 60000)
print(emp.name)          # Public accessible
emp.show_salary()        # Accessing private correctly
# print(emp._ _salary)    # Error: Not accessible directly
```

Explanation:

self._ _salary: Defined with double underscores, so it is private.

show_salary(): A public method that provides safe access to the private attribute.

Attempting **emp._ _salary** causes an AttributeError, proving private members cannot be accessed directly.

Declaring Protected and Private Methods

In Python, you can control method access levels using naming conventions:

- Use a single underscore (_) before a method name to indicate it is protected meant to be used within class or its subclasses.
- Use a double underscore (_ _) to define a private method accessible only within class due to name mangling.

Example: This example demonstrates how a **protected method (_show_balance)** and a **private method (__update_balance)** are used to control access. The private method updates balance internally, while protected method displays it. Both are accessed via a public method (deposit), showing how Python uses naming conventions for encapsulation.

```
class BankAccount:
    def __init__(self):
        self.balance = 1000

    def _show_balance(self):
        print(f"Balance: ₹{self.balance}")  # Protected method

    def __update_balance(self, amount):
        self.balance += amount              # Private method

    def deposit(self, amount):
        if amount > 0:
            self.__update_balance(amount)  # Accessing private method internally
            self._show_balance()           # Accessing protected method
        else:
            print("Invalid deposit amount!")

account = BankAccount()
account._show_balance()      # Works, but should be treated as internal
# account.__update_balance(500)  # Error: private method
account.deposit(500)         # Uses both methods internally
```

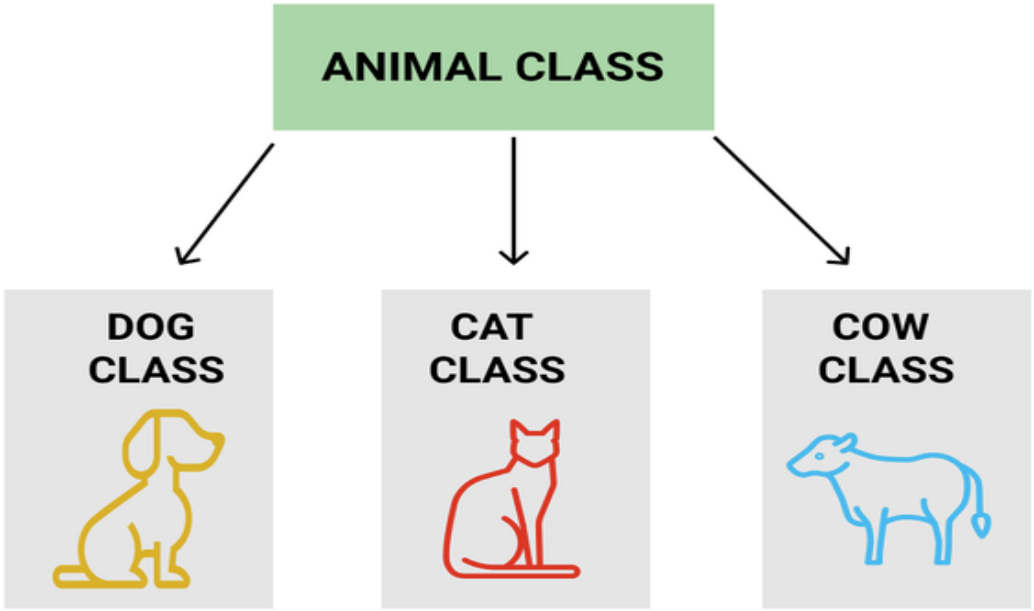
Getter and Setter Methods

- In Python, **getter** and **setter** methods are used to access and modify private attributes safely. I
- nstead of accessing private data directly, these methods provide controlled access, allowing you to:
- Read data using a getter method.
- Update data using a setter method with optional validation or restrictions.

```
class Employee:
    def _ _init_ _(self):
        self._ _salary = 50000  # Private attribute

    def get_salary(self):    # Getter method
        return self._ _salary

    def set_salary(self, amount):   # Setter method
        if amount > 0:
            self._ _salary = amount
        else:
            print("Invalid salary amount!")

emp = Employee()
print(emp.get_salary())  # Access salary using getter

emp.set_salary(60000)   # Update salary using setter
print(emp.get_salary())
```

Inheritance

Inheritance allows a class (child class) to acquire properties and methods of another class (parent class). It supports hierarchical classification and promotes code reuse.



- Inheritance allows us to define a class that inherits all the methods and properties from another class.
- **Parent class** is the class being inherited from, also called base class.
- **Child class** is the class that inherits from another class, also called derived class.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class:

Example

Create a class named `Person`, with `firstname` and `lastname` properties, and a `printname` method:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

#Use the Person class to create an object, and then execute the printname method:

x = Person("John", "Doe")
x.printname()
```

Create a Child Class

To create a class that inherits the functionality from another class, send the parent class as a parameter when creating the child class:

Example

Create a class named `Student`, which will inherit the properties and methods from the `Person` class:

```
class Student(Person):
  pass
```

Note: Use the `pass` keyword when you do not want to add any other properties or methods to the class.

Now the Student class has the same properties and methods as the Person class.

Example

Use the `Student` class to create an object, and then execute the `printname` method:

```
x = Student("Mike", "Olsen")
x.printname()
```

Add the __init__() Function

So far we have created a child class that inherits the properties and methods from its parent.

We want to add the `__init__()` function to the child class (instead of the `pass` keyword).

Note: The `__init__()` function is called automatically every time the class is being used to create a new object.

Example

Add the `__init__()` function to the `Student` class:

```
class Student(Person):
  def __init__(self, fname, lname):
    #add properties etc.
```

When you add the `__init__()` function, the child class will no longer inherit the parent's `__init__()` function.

Note: The child's `__init__()` function **overrides** the inheritance of the parent's `__init__()` function.

To keep the inheritance of the parent's __init__() function, add a call to the parent's __init__() function:

```
class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
```

Now we have successfully added the `__init__()` function, and kept the inheritance of the parent class, and we are ready to add functionality in the `__init__()` function.

Use the super() Function:

Python also has a super() function that will make the child class inherit all the methods and properties from its parent:

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
```

By using the `super()` function, you do not have to use the name of the parent element, it will automatically inherit the methods and properties from its parent.

Add Properties

Example

Add a property called `graduationyear` to the `Student` class:

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
    self.graduationyear = 2019
```

In the example below, the year `2019` should be a variable, and passed into the `Student` class when creating student objects. To do so, add another parameter in the `__init__()` function:

Add a `year` parameter, and pass the correct year when creating objects:

```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

x = Student("Mike", "Olsen", 2019)
```

Add Methods

Example

Add a method called `welcome` to the `Student` class:

```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)
```

If you add a method in the child class with the same name as a function in the parent class, the inheritance of the parent method will be overridden.

Example:

```
class Engine:
  a=10
  def __init__(self):
   self.b=20
  def m1(self):
   print("Engine Specific Functionality")
class Car:
  def __init__(self):
   self.engine=Engine()
  def m2(self):
   print("Car using Engine class functionality")
   print(self.engine.a)
   print(self.engine.b)
   self.engine.m1()
c=Car()
c.m2()
```

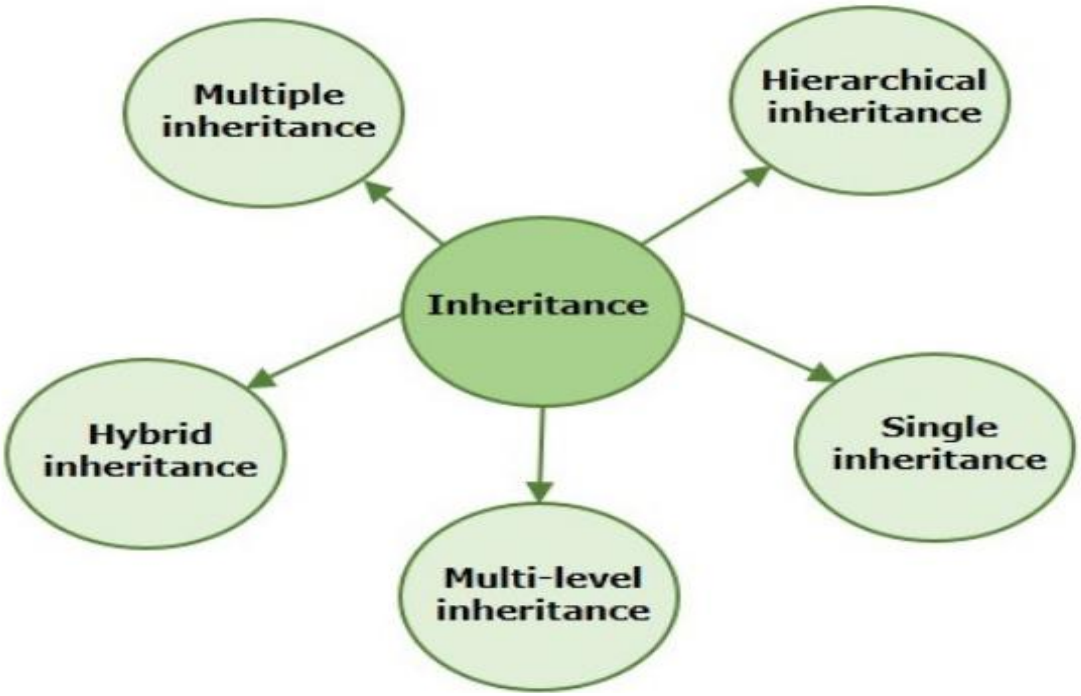
Output:

```
Car using Engine class functionality
10
20
Engine Specific Functionality
```

Types of Inheritance in Python

In Python, inheritance can be divided in five different categories –

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

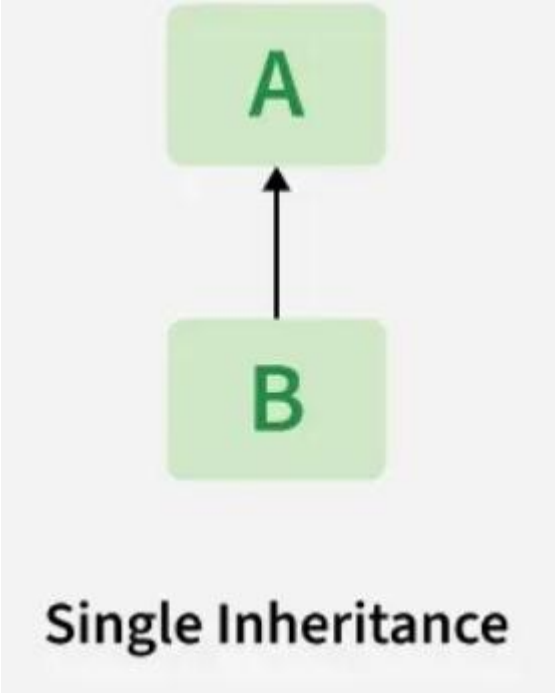


Single Inheritance

This is the simplest form of inheritance where a child class inherits attributes and methods from only one parent class.

Example:

```
# parent class
class Parent:
   def parentMethod(self):
      print ("Calling parent method")

# child class
class Child(Parent):
   def childMethod(self):
      print ("Calling child method")

# instance of child
c = Child()
# calling method of child class
c.childMethod()
# calling method of parent class
c.parentMethod()
```

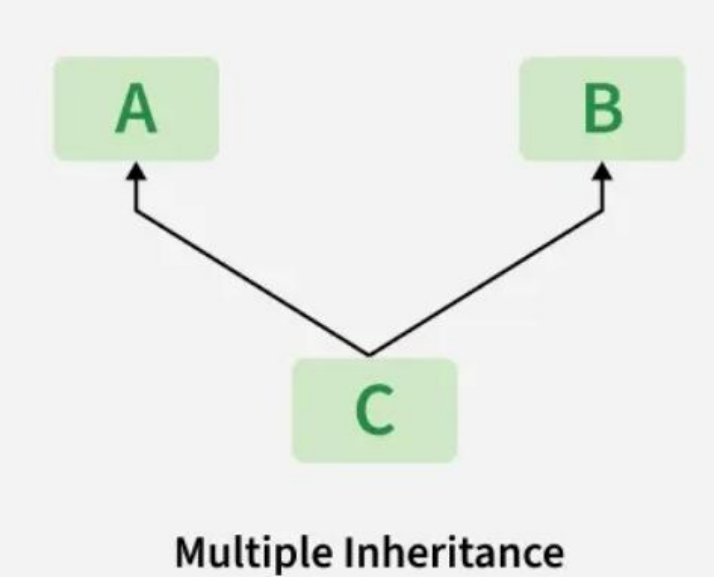


Multiple Inheritance

Multiple inheritance in Python allows you to construct a class based on more than one parent classes. The Child class thus inherits the attributes and method from all parents. The child can override methods inherited from any parent.

Syntax:

```
class parent1:

   #statements

class parent2:

   #statements

class child(parent1, parent2):

   #statements
```



Multiple Inheritance

Example

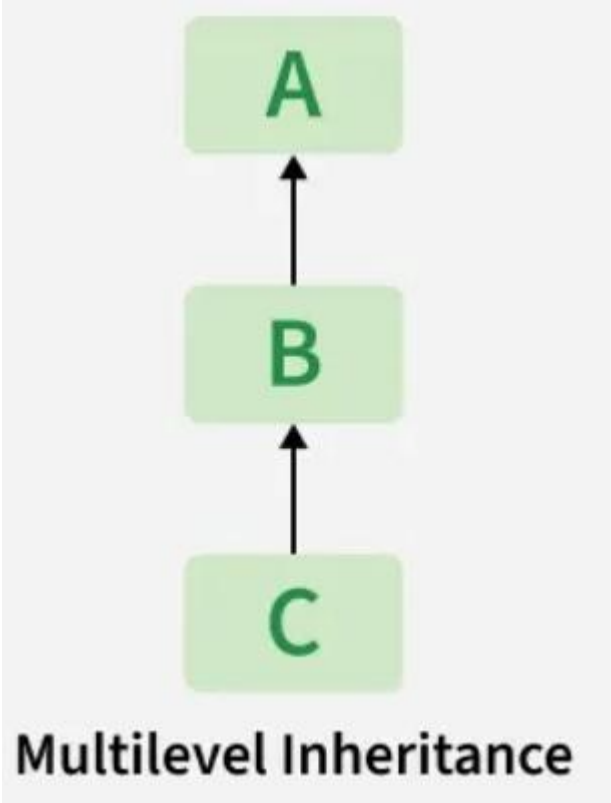
```
# Base class 1
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

# Base class 2
class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

# Derived class
class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)
# Driver code
s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

Multilevel Inheritance

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and a grandfather.

Example:

```
# parent class
class Universe:
  def universeMethod(self):
    print ("I am in the Universe")

# child class
class Earth(Universe):
  def earthMethod(self):
    print ("I am on Earth")

# another child class
class India(Earth):
  def indianMethod(self):
    print ("I am in India")

# creating instance
person = India()
# method calls
person.universeMethod()
person.earthMethod()
person.indianMethod()
```

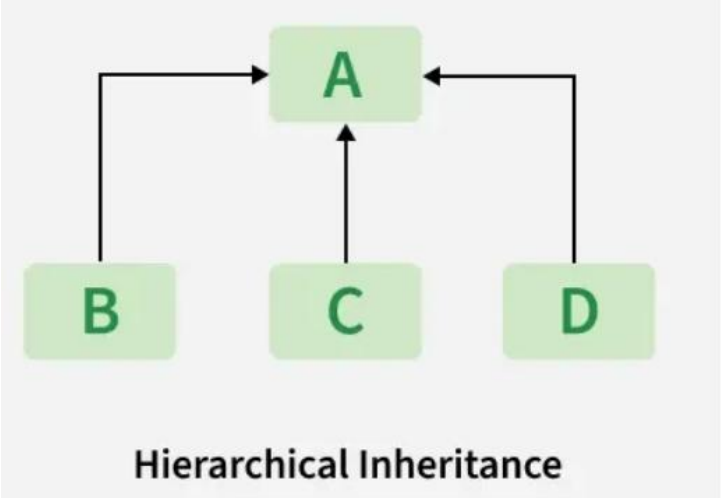


Hierarchical Inheritance

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.

Example:

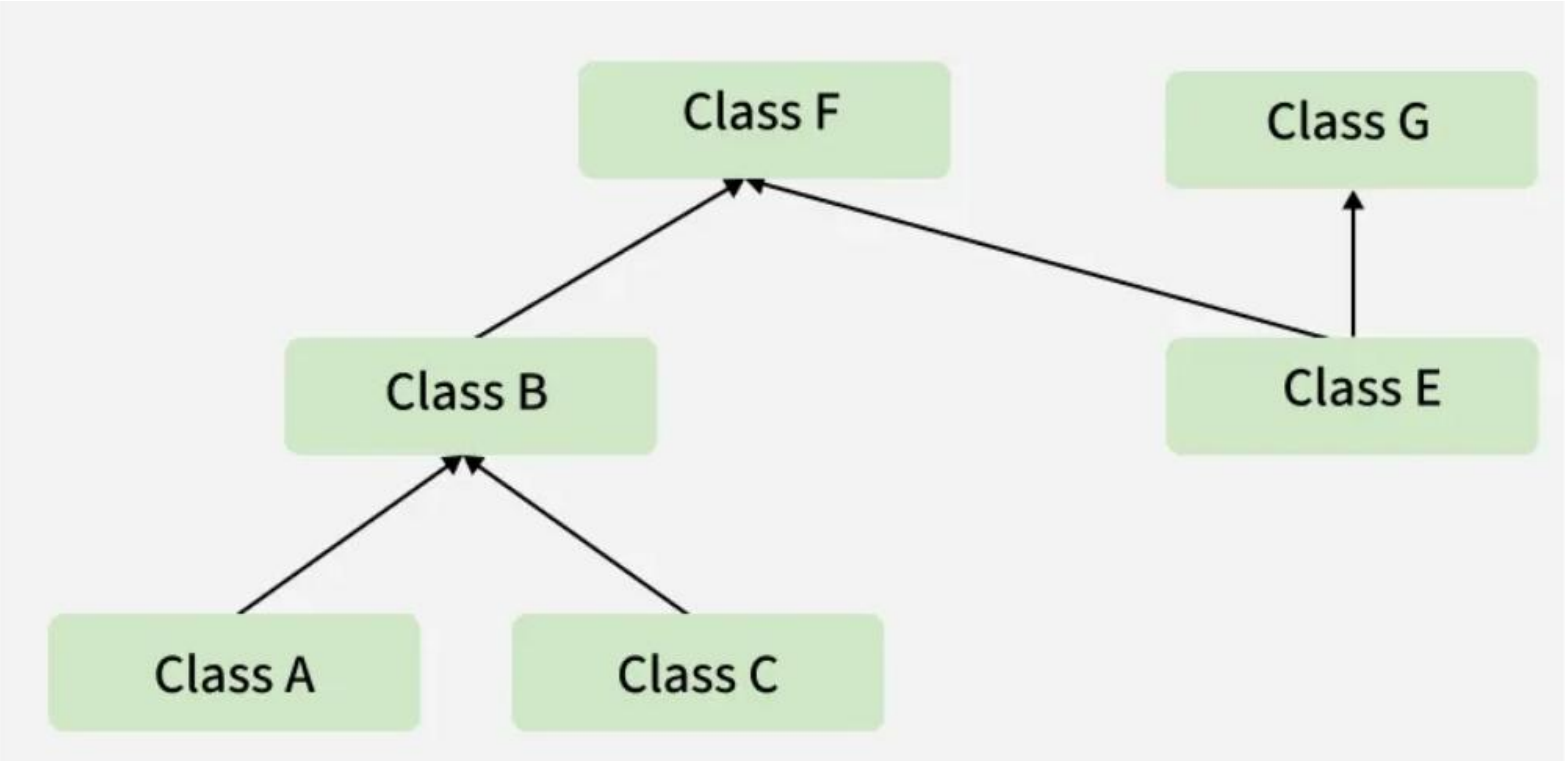
```
# parent class
class Manager:
  def managerMethod(self):
    print ("I am the Manager")

# child class
class Employee1(Manager):
  def employee1Method(self):
    print ("I am Employee one")

# second child class
class Employee2(Manager):
  def employee2Method(self):
    print ("I am Employee two")

# creating instances
emp1 = Employee1()
emp2 = Employee2()
# method calls
emp1.managerMethod()
emp1.employee1Method()
emp2.managerMethod()
emp2.employee2Method()
```



Hierarchical Inheritance

Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance. It uses a mix like single, multiple, or multilevel inheritance within the same program. Python's method resolution order (MRO) handles such cases.



Example

```
# parent class
class CEO:
   def ceoMethod(self):
      print ("I am the CEO")

class Manager(CEO):
   def managerMethod(self):
      print ("I am the Manager")

class Employee1(Manager):
   def employee1Method(self):
      print ("I am Employee one")

class Employee2(Manager, CEO):
   def employee2Method(self):
      print ("I am Employee two")

# creating instances
emp = Employee2()
# method calls
emp.managerMethod()
emp.ceoMethod()
emp.employee2Method()
```

The super() function

In Python, super() function allows you to access methods and attributes of the parent class from within a child class.

Example:

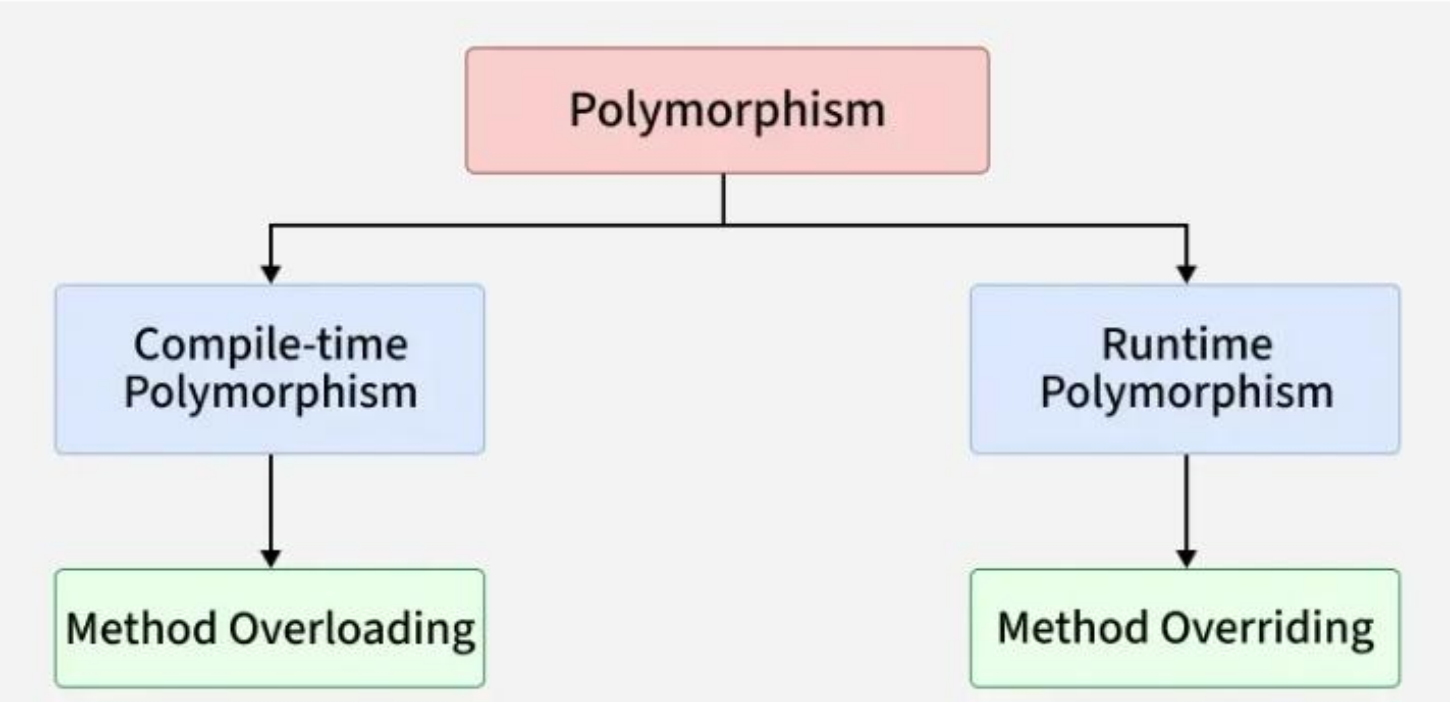
```
# parent class
class ParentDemo:
   def __init__(self, msg):
      self.message = msg

   def showMessage(self):
      print(self.message)

# child class
class ChildDemo(ParentDemo):
   def __init__(self, msg):
      # use of super function
      super().__init__(msg)

# creating instance
obj = ChildDemo("Welcome to Tutorialspoint!!")
obj.showMessage()
```

Polymorphism

- Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.
- Technically, in Python, polymorphism allows same method, function or operator to behave differently depending on object it is working with. This makes code more flexible and reusable.
- **Real Life Example:** In a backend payment system, multiple payment options are available such as Credit Card, UPI, NetBanking and Wallet. All payment types use a common method named processPayment() but different implementations:
  - Credit Card Payment: validates card, talks to bank API
  - UPI Payment: redirects to UPI gateway
  - Wallet Payment: checks wallet balance
  - NetBanking Payment: redirects to bank login
- The method name remains the same, but the action changes based on the payment type.

Types of Polymorphism

- Polymorphism in Python refers to ability of the same method or operation to behave differently based on object or context. It mainly includes compile-time and runtime polymorphism.



1. Compile-time Polymorphism

- Compile-time polymorphism means deciding which method or operation to run during compilation, usually through method or operator overloading. Languages like Java or C++ support this.
- But Python doesn't because it's dynamically typed it resolves method calls at runtime, not during compilation.
- So, true method overloading isn't supported in Python, though similar behavior can be achieved using default or variable arguments.

Example: This code demonstrates method overloading in Python using default and variable-length arguments. The multiply() method works with different numbers of inputs, mimicking compile-time polymorphism.

```python
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

# Create object
calc = Calculator()

# Using default arguments
print(calc.multiply())
print(calc.multiply(4))

# Using multiple arguments
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
```

2. Runtime Polymorphism (Overriding)

- Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it.
- In Python, this happens through **Method Overriding** a child class provides its own version of a method already defined in the parent class. Since Python is dynamic, it supports this, allowing same method call to behave differently for different object types.

Example: This code shows runtime polymorphism using method overriding. The sound() method is defined in base class Animal and overridden in Dog and Cat. At runtime, correct method is called based on object's class.

```
class Animal:
    def sound(self):
        return "Some generic sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

# Polymorphic behavior
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

Polymorphism in Built-in Functions

Python's built-in functions like **len()** and **max()** are polymorphic they work with different data types and return results based on type of object passed. This showcases it's dynamic nature, where same function name adapts its behavior depending on input.

Example: This code demonstrates polymorphism in Python's built-in functions handling strings, lists, numbers and characters differently while using same function name.

```
print(len("Hello"))  # String length
print(len([1, 2, 3]))  # List length

print(max(1, 3, 2))  # Maximum of integers
print(max("a", "z", "m"))  # Maximum in strings
```

Output

```
5
3
3
z
```

Polymorphism in Functions

In Python, polymorphism lets functions accept different object types as long as they support needed behavior. Using duck typing, Python focuses on whether an object has right method not its type allowing flexible and reusable code.

Example: This code demonstrates polymorphism using duck typing as perform_task() function works with different object types (Pen and Eraser), as long as they have a **.use()** method showing flexible and reusable function design.

```
class Pen:
    def use(self):
        return "Writing"

class Eraser:
    def use(self):
        return "Erasing"

def perform_task(tool):
    print(tool.use())

perform_task(Pen())
perform_task(Eraser())
```

Output

```
Writing
Erasing
```

Polymorphism in Operators

In Python, same operator (+) can perform different tasks depending on operand types. This is known as operator overloading. This flexibility is a key aspect of polymorphism in Python.

Example: This code shows operator polymorphism as + operator behaves differently based on data types adding integers, concatenating strings and merging lists all using same operator.

```
print(5 + 10)  # Integer addition
print("Hello " + "World!")  # String concatenation
print([1, 2] + [3, 4])  # List concatenation
```

Output

```
15
Hello World!
[1, 2, 3, 4]
```

Data Abstraction in Python

Data abstraction means showing only the essential features and hiding the complex internal details. In Python, abstraction is used to hide the implementation details from the user and expose only necessary parts, making the code simpler and easier to interact with.

Real Life Example: In a graphics software, multiple shapes are available such as Circle, Rectangle and Triangle. All shapes share common data and support the same set of actions, but the internal details are hidden:

- **Common data:** name, color, border thickness, fill style.
- **Common actions:** draw(), resize(), getArea().
- Each shape implements these actions differently Circle uses radius, Rectangle uses length and width and Triangle uses base and height.

The software only knows that a shape can be drawn and its area can be calculated. How the shape is drawn or how the area is calculated is hidden.

Abstract Base Class

In Python, an Abstract Base Class (ABC) is used to achieve data abstraction by defining a common interface for its subclasses. It cannot be instantiated directly and serves as a blueprint for other classes.

Abstract classes are created using abc module and @abstractmethod decorator, allowing developers to enforce method implementation in subclasses while hiding complex internal logic.

```
from abc import ABC, abstractmethod

class Greet(ABC):
    @abstractmethod
    def say_hello(self):
        pass  # Abstract method

class English(Greet):
    def say_hello(self):
        return "Hello!"

g = English()
print(g.say_hello())
```

Output

```
Hello!
```

Components of Abstraction

Abstraction in Python is made up of key components like abstract methods, concrete methods, abstract properties and class instantiation rules. These elements work together to define a clear and enforced structure for subclasses while hiding unnecessary implementation details. Let's discuss them one by one.

Abstract Method

Abstract methods are method declarations without a body defined inside an abstract class. They act as placeholders that force subclasses to provide their own specific implementation, ensuring consistent structure across derived classes.

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass  # Abstract method, no implementation here
```

Explanation: make_sound() is an abstract method in Animal class, so it doesn't have any code inside it.

Concrete Method

Concrete methods are fully implemented methods within an abstract class. Subclasses can inherit and use them directly, promoting code reuse without needing to redefine common functionality.

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass  # Abstract method, to be implemented by subclasses

    def move(self):
        return "Moving"  # Concrete method with implementation
```

Explanation: move() method is a concrete method in Animal class. It is implemented and does not need to be overridden by Dog class.

Abstract Properties

Abstract properties work like abstract methods but are used for properties. These properties are declared with @property decorator and marked as abstract using @abstractmethod. Subclasses must implement these properties.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @property
    @abstractmethod
    def species(self):
        pass  # Abstract property, must be implemented by subclasses

class Dog(Animal):
    @property
    def species(self):
        return "Canine"

# Instantiate the concrete subclass
dog = Dog()
print(dog.species)
```

Output

```
Canine
```

Abstract Class Instantiation

Abstract classes cannot be instantiated directly. This is because they contain one or more abstract methods or properties that lack implementations. Attempting to instantiate an abstract class results in a TypeError.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

animal = Animal()
```

Explanation:

- Animal class is abstract because it has make_sound() method as an abstract method.
- Instantiating Animal() raises a TypeError because abstract classes with unimplemented methods can't be instantiated, only fully implemented subclasses can.

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI